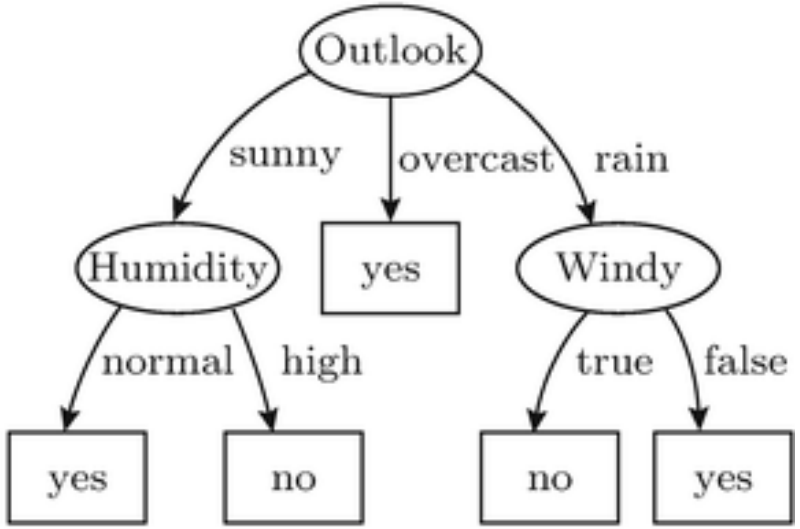
Decision Tree

Introduction to Decision Tree

- Like SVMs, *Decision Trees* are versatile Machine Learning algorithms that can perform both classification and regression tasks, and even multioutput tasks.
- They are very powerful algorithms, capable of fitting complex datasets.
- Decision Trees are also the fundamental components of Random Forests, which are among the most powerful Machine Learning algorithms available today.
- We will mainly go through the classification tasks using decision tree
- Unlike various other ML models where it is difficult to understand the model, Decision tree is Simple to understand and to interpret. Trees can be visualized.
- Also, it works for both continuous and categorical attributes!
- And you don’t need scale the data

Example look of a decision tree

Outlook	*Temp*	*Humidity*	*Windy*	*Golf?*
rainy	hot	high	false	no
rainy	hot	high	true	no
overcast	hot	high	false	yes
sunny	mild	high	false	yes
sunny	cool	normal	false	yes
sunny	cool	normal	true	no
overcast	cool	normal	true	yes
rainy	mild	high	false	no
rainy	cool	normal	false	yes
sunny	mild	normal	false	yes
rainy	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
sunny	mild	high	true	no

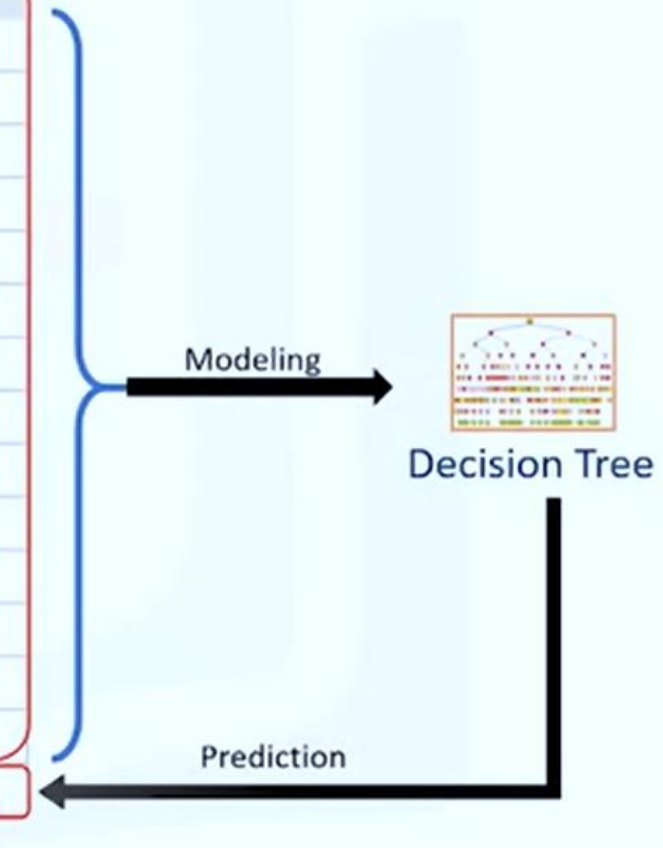


You can easily come up with a set of if-else rules from the decision tree such as:
If outlook = overcast then golf = yes
Else If outlook = sunny AND Humidity = normal then golf = yes
And so on……

Building a Decision Tree

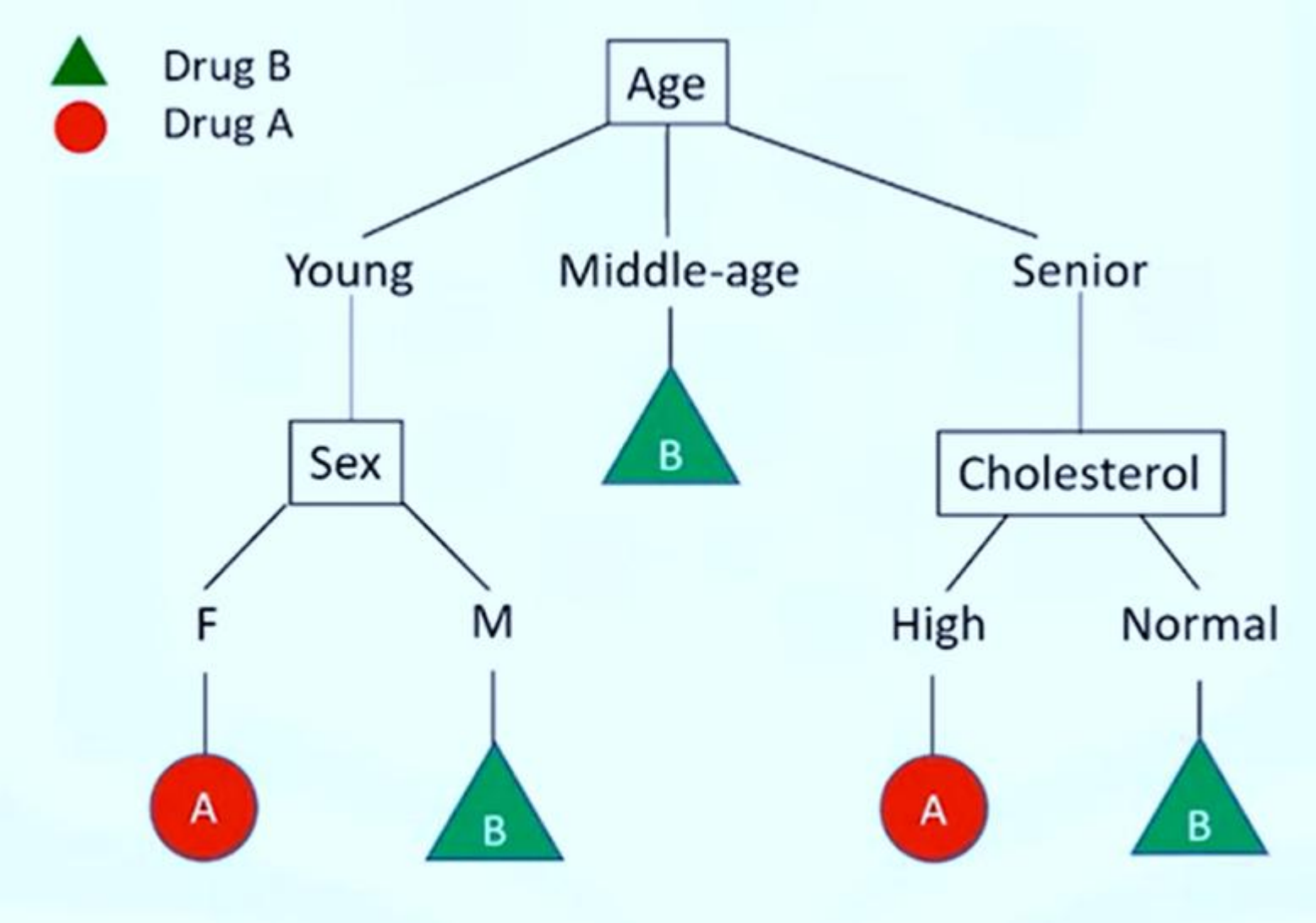
- Let us consider the following data set where we have some attributes of patients who suffered from the same disease, and we have the record on which medicine were effective for them.
- We want to learn a decision tree model from the data set so that we can predict the effective medicine for new patient.

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A
p15	Middle-age	F	Low	Normal	?



Building a decision tree based on the training set

- We want to prescribe a drug to a penitent
- The decision depends on the patient's situation and demographic based on our training set.
- From the decision tree, if you see if the person is in middle-age, drug B is most effective without knowing other attributes.
- However, if the person is young, to make decision we need to know the gender.
- If you see the tree:
  - Each internal node (e.g., age, sex, and cholesterol) is a condition
  - Each branch from root to the leaf is a decision rule
  - The leaf node assigns a classification



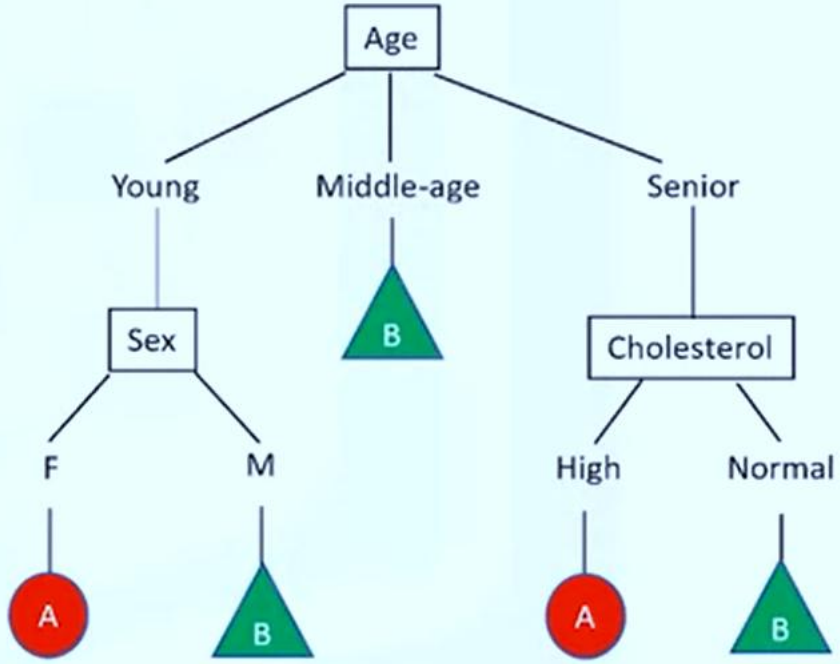
Steps of learning a decision tree

1. Choose an attribute from the data set
2. Calculate the significance of the attribute in splitting of the data
    1. We will see more about it soon
3. Split the data based on the value of the best attribute
4. Go to step 1

Building a decision tree

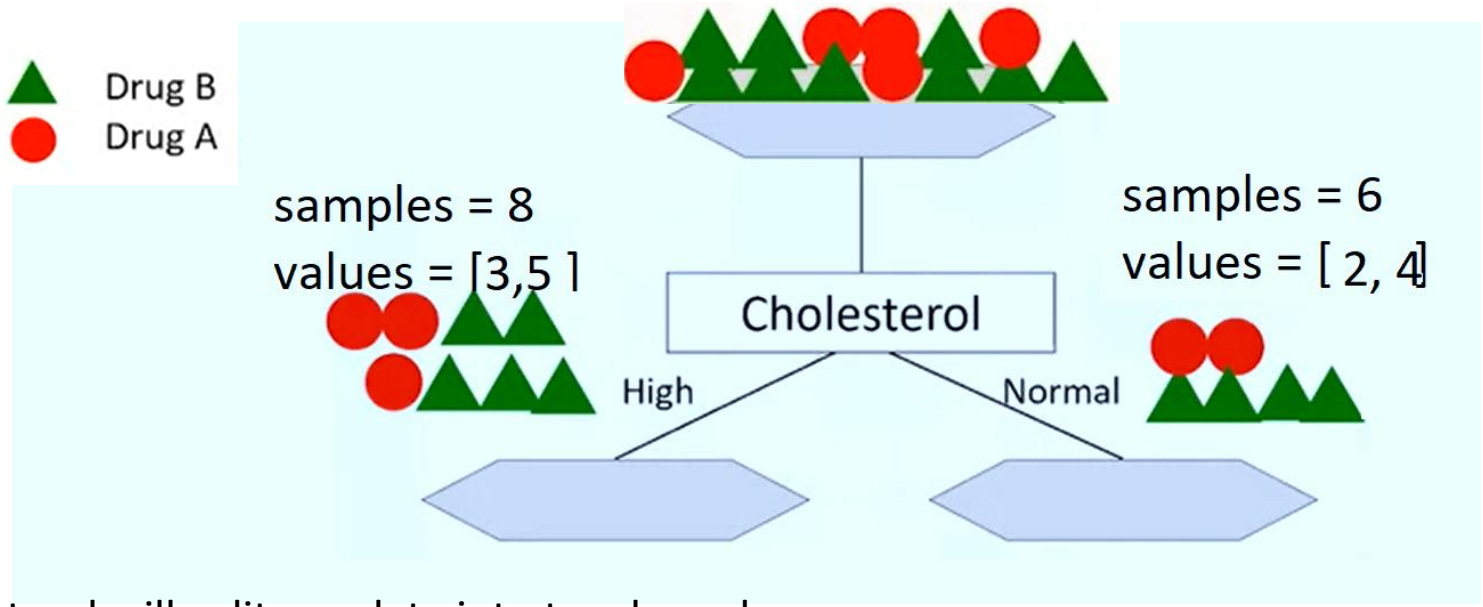
- So, we want to build a decision tree as shown on the right side of the following figure based on the data set shown on the left side following the steps mentioned in the last slide

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A
p15	Middle-age	F	Low	Normal	?

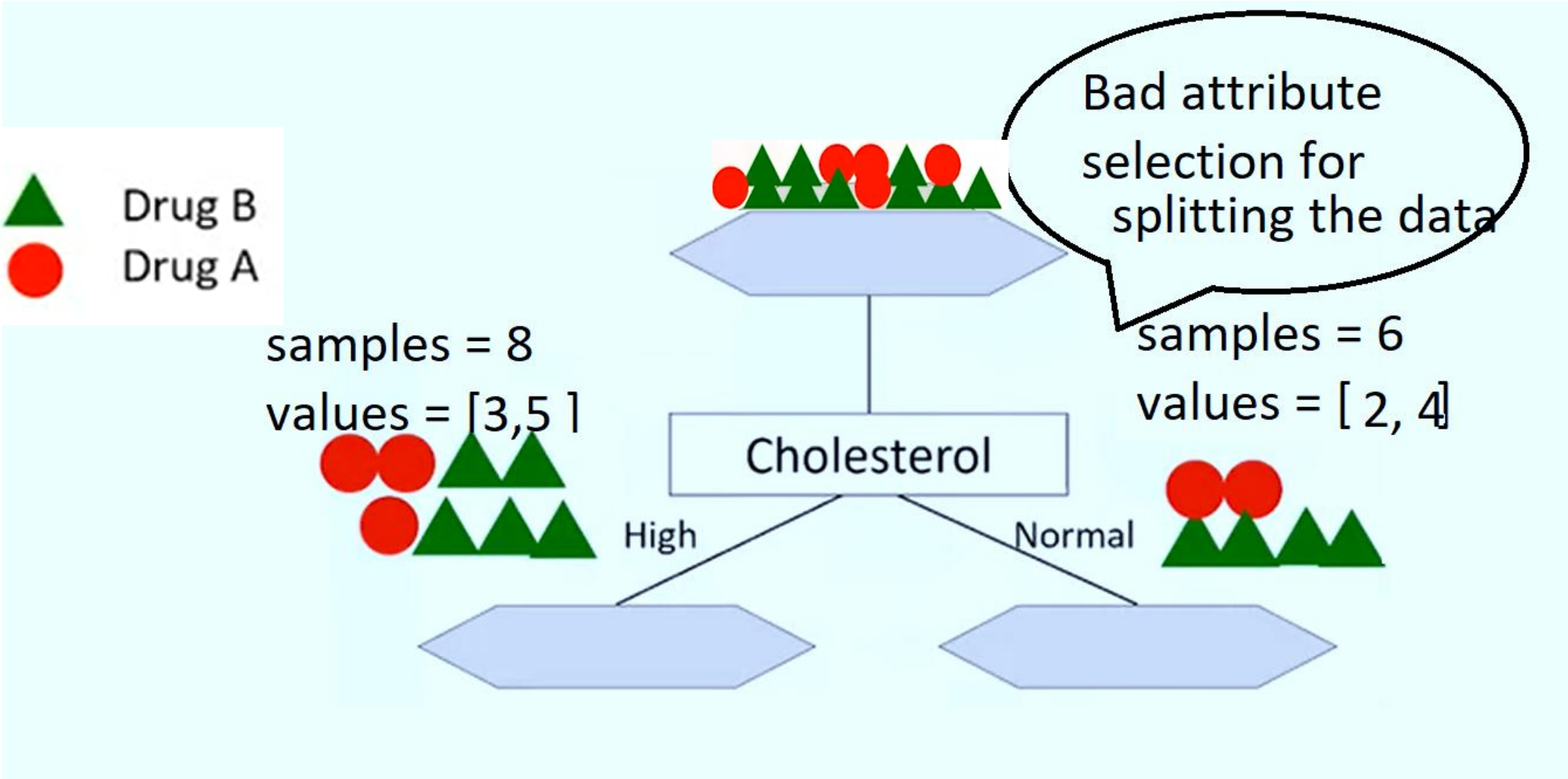


Choosing the best attribute for partitioning

- Assume we have 14 instances and 5 features (age, sex, BP, and Cholesterol)
- Let us choose cholesterol and see whether it is a good selection or not *(Note that this number will not match the table shown earlier. This is just to give you an idea at this moment)*

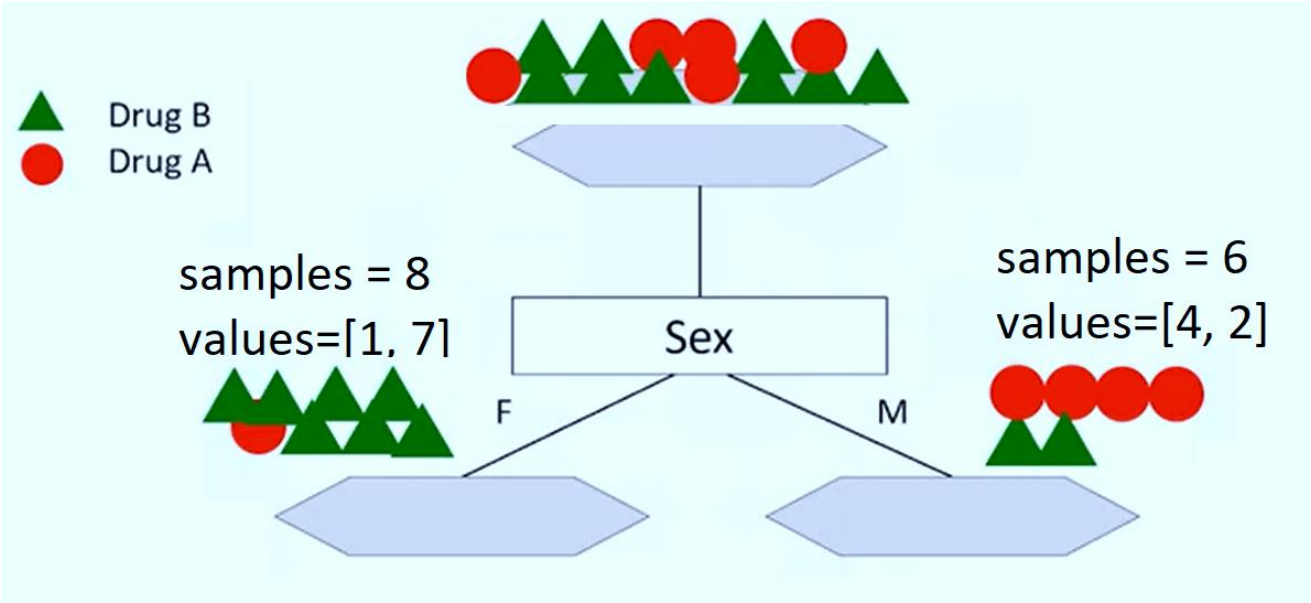


- Choosing Cholesterol will split our data into two branches
- Based on this, you might say that if the Cholesterol is high, we should recommend Drug B. [However, our confidence will be very low]
- Same issue with the Normal Cholesterol. In fact, both cases Drug B is the majority.]
- So, we don't have sufficient evidence or information to determine drug A or B just based on cholesterol
- It seems Splitting the Cholesterol is **not a good choice**


Drug B
Drug A
Bad attribute selection for splitting the data
samples = 8
values = [3,5]
samples = 6
values = [2, 4]
Cholesterol
High
Normal

Choosing the best attribute for partitioning

- We have 14 instances and 5 features (age, sex, BP, and Cholesterol)
- Let us choose Sex attribute and see whether is it a good selection or not *(Note that this number will not match the table shown earlier. This is just to give you an idea at this moment)*



- Much better than Cholesterol
- More predictiveness
- Less Impurity
- Lower Entropy

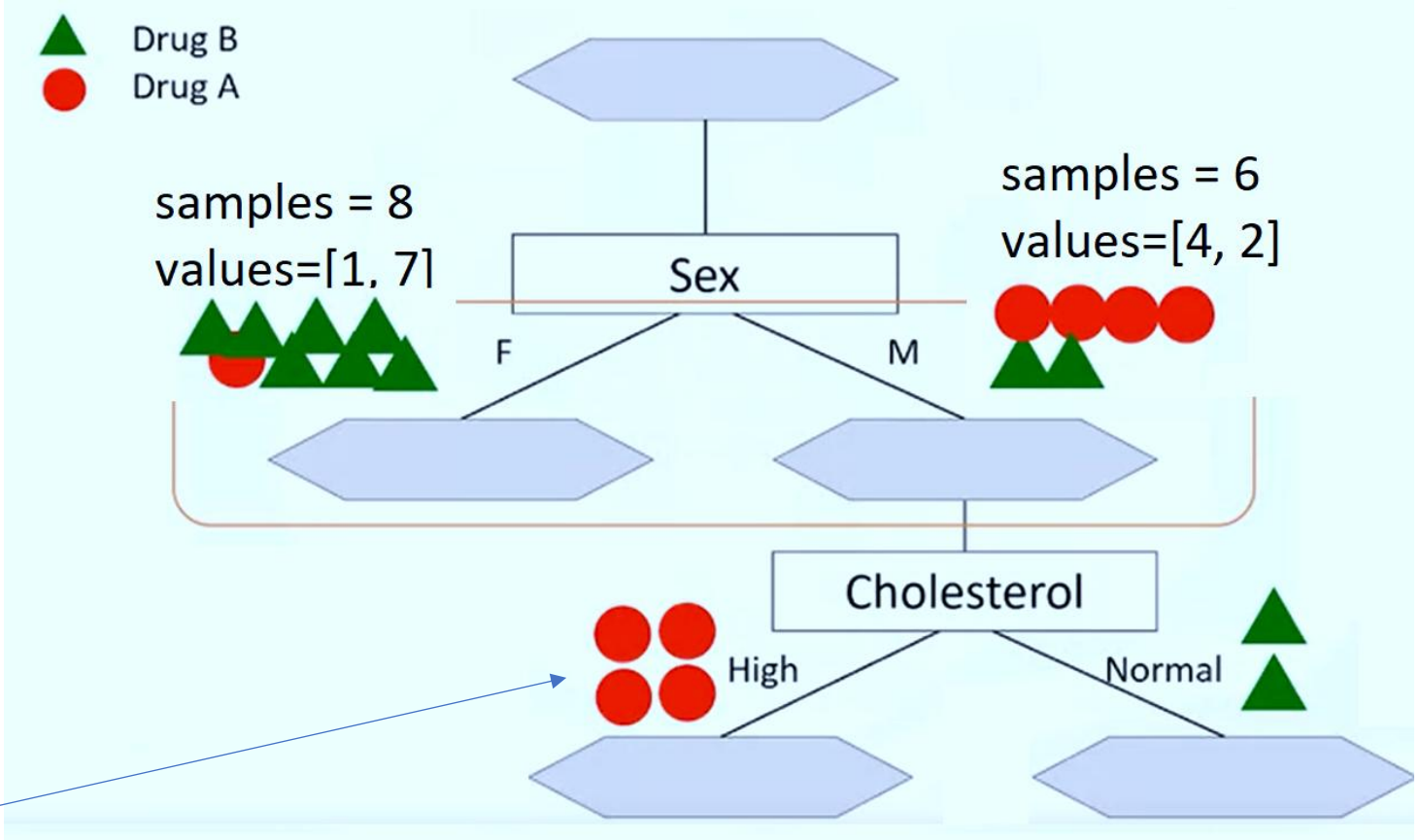
We will discuss more about Impurity and entropy soon

- For a female we are quite confident, but for M we may not be that confident
- However, sex attribute is a better choice (mor predictiveness) compared to Cholesterol
- Predictiveness is based on decrease in impurity of nodes
- We are looking for the best feature to decrease the impurity of patients in the leaves
- So, the sex feature is a good candidate in the following case because it almost found the pure patients.

Choosing the best attribute for partitioning

- Let us split more the M patient branch

- It results in more pure leaves, and we can easily make decision
- So, we can confidently prescribe drug A if a patient is M and have high cholesterol
- Similarly, prescribe drug B if sex = 'M' and Cholesterol = 'Normal'



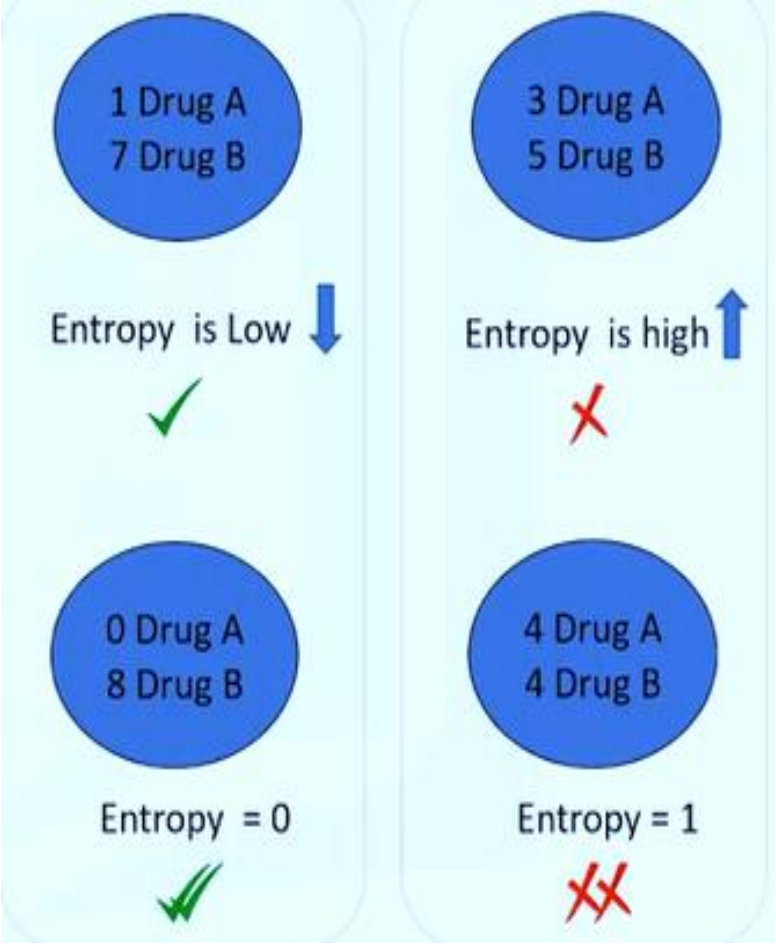
- So, we can see choice of attribute is very important and is all about purity of the leaves after the split

A node pure if in 100 percent of the cases, the nodes fall into a specific category of the target column.

The algorithm uses recursive partitioning to split the training records into segments by minimizing the impurity at each step.

Calculating Impurity and Entropy

- The impurity of a node is calculated by the Entropy
- Entropy:
  - It is a measurement of randomness or uncertainty
  - So, we want less uncertainty, it means we want low entropy
  - Lower the entropy, the purer the node
  - Lower the entropy in a node, more homogeneous the samples



$$Entropy\ (p) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

For our case: Entropy = $-p(A)\log_2(p(A)) - p(B)\log_2(p(B))$

For our case: Entropy = $-p(A)\log_2(p(A)) - p(B)\log_2(p(B))$

Calculate Entropy

- **Let us calculate Entropy before splitting**

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A

S: [9 B, 5 A]

$E = -p(B)\log(p(B)) - p(A)\log(p(A))$

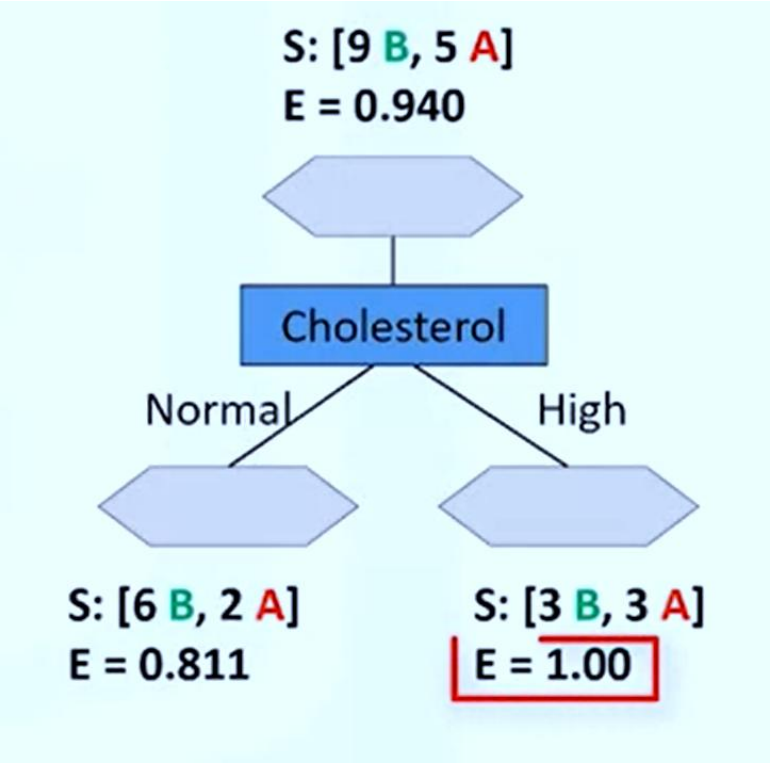
$E = -(9/14)\log(9/14) - (5/14)\log(5/14$

$E = 0.940$

Now Choose Cholesterol and Calculate Entropy

For our case: Entropy = $-p(A)\log_2(p(A)) - p(B)\log_2(p(B))$

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A

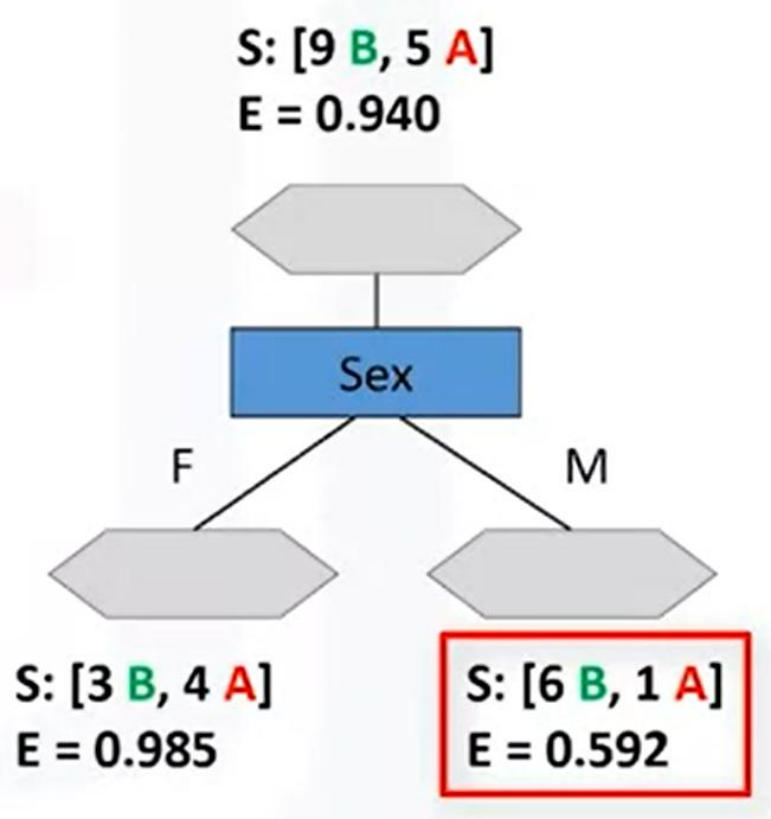


We need to calculate Entropy based on splitting all the attribute and choose the best on

Now Choose Sex attribute and Calculate Entropy

For our case: Entropy = $-p(A)\log_2(p(A)) - p(B)\log_2(p(B))$

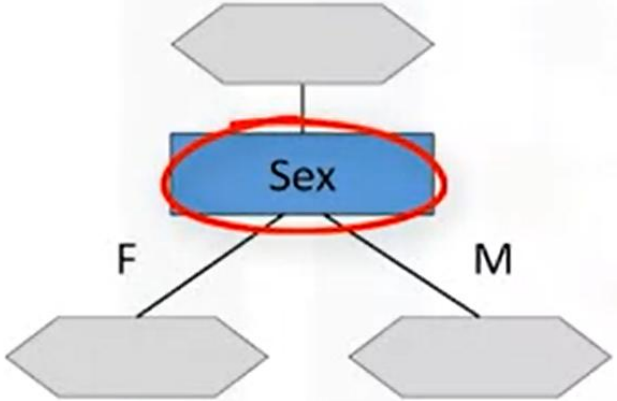
Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A



We need to calculate Entropy based on splitting all the attribute and choose the best one. For now, just decide between Sex and Cholesterol attributes

Should we split by Cholesterol or Sex attribute?

S: [9 B, 5 A]
E = 0.940



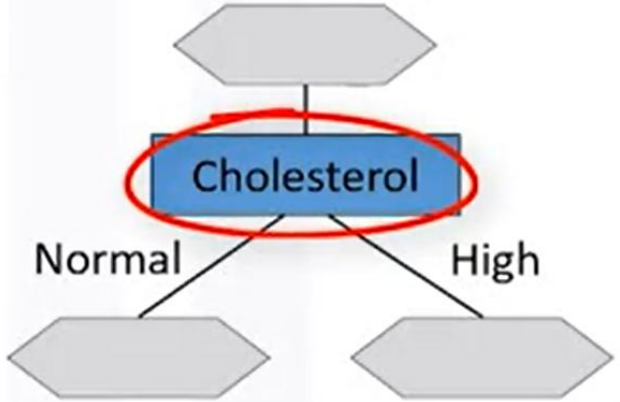
S: [3 B, 4 A]
E = 0.985

S: [6 B, 1 A]
E = 0.592

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	Hiigh	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A

?

S: [9 B, 5 A]
E = 0.940



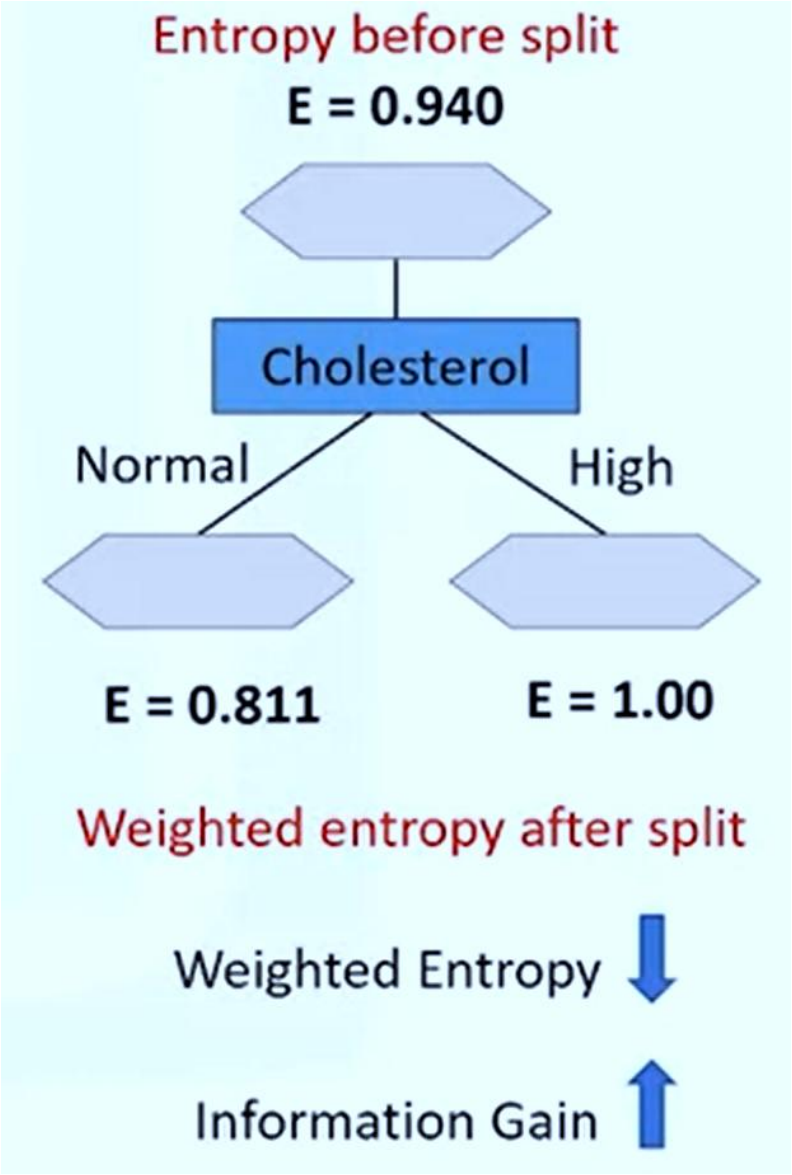
S: [6 B, 2 A]
E = 0.811

S: [3 B, 3 A]
E = 1.00

- Which one is better at the first attribute to divide the data-set into two branches?
- Or in other words, which attribute results in more pure nodes for our drugs?
- We need to know the Information Gain After Splitting and based on that we can decide

Information Gain

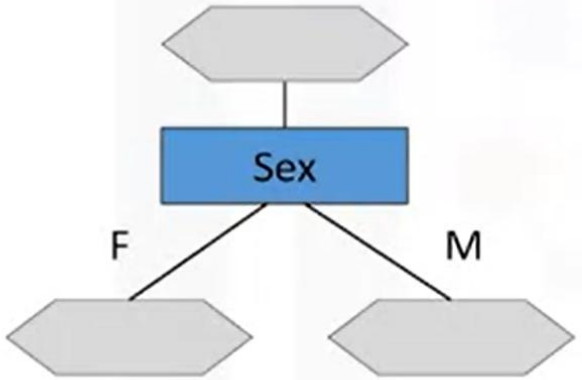
- It is the in information that can increase the level of certainty after splitting
- **Information Gain = (Entropy before split) – (Weighted entropy after split)**
- **As the entropy increases, the amount of information gain decreases**
- **On the other hand, if entropy decreases, information gain or the amount of certainly increases**



Let us calculate information gain

Information Gain = (Entropy before split) – (Weighted entropy after split)

S: [9 B, 5 A]
E = 0.940



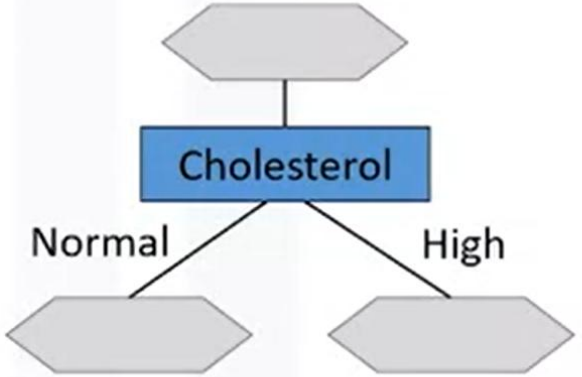
S: [3 B, 4 A]
E = 0.985

S: [6 B, 1 A]
E = 0.592

Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	High	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A

?

S: [9 B, 5 A]
E = 0.940



S: [6 B, 2 A]
E = 0.811

S: [3 B, 3 A]
E = 1.00

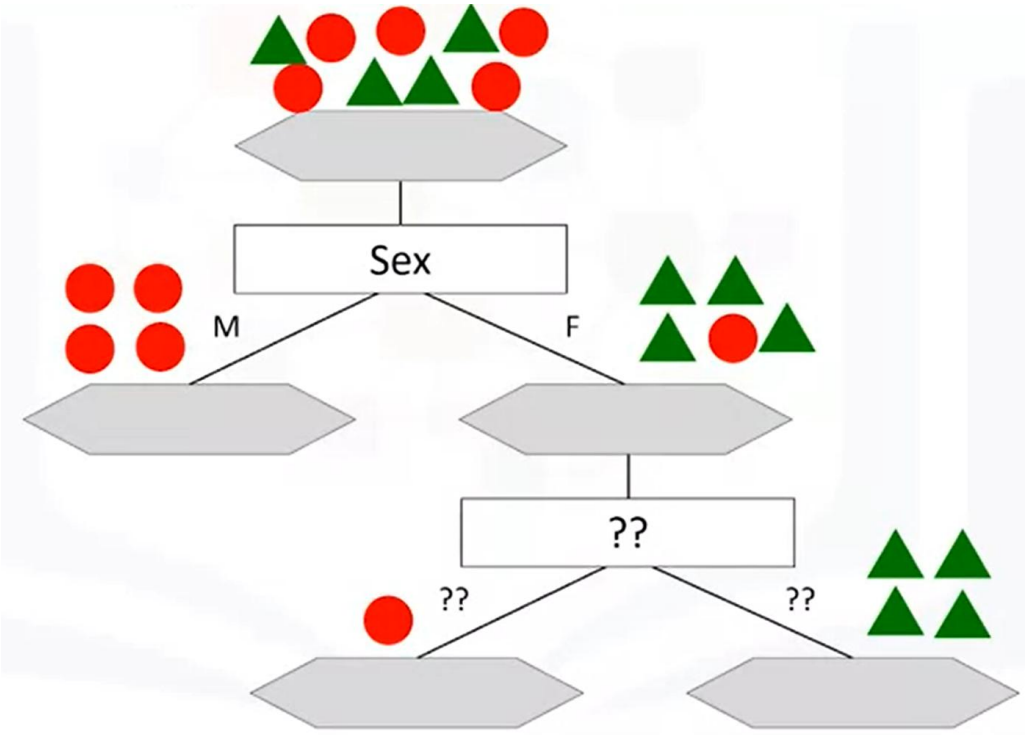
Gain (s, Sex)
= 0.940 – [(7/14)0.985 + (7/14)0.592]
=0.151

Gain (s, Cholesterol)
= 0.940 – [(8/14).811 + (6/14)1.0]
=0.048

So, it looks like we got highest information gain by splitting sex attribute

Assuming we have only two attributes (Sex and Cholesterol), we have chosen to split by the sex attribute

- *(However, note that in practical case, , you need to try with all the 4 attributes to find which attribute gives the highest information gain)*
- Now, we have to keep doing it recursively for further splitting



Decision Tree Algorithms and scikit-learn

- **Tree algorithms: ID3, C4.5, C5.0 and CART**
- What are all the various decision tree algorithms and how do they differ from each other? Which one is implemented in scikit-learn?
- ID3 (Iterative Dichotomiser 3) was developed in 1986 by Ross Quinlan. The algorithm creates a multiway tree, finding for each node (i.e. in a greedy manner) the *categorical feature* that will yield the largest information gain for categorical targets. Trees are grown to their maximum size and then a pruning step is usually applied to improve the ability of the tree to generalize to unseen data.
- C4.5 is the successor to ID3 and removed the restriction that features must be categorical by dynamically defining a discrete attribute (based on numerical variables) that partitions the continuous attribute value into a discrete set of intervals. C4.5 converts the trained trees (i.e. the output of the ID3 algorithm) into sets of if-then rules. The accuracy of each rule is then evaluated to determine the order in which they should be applied. Pruning is done by removing a rule's precondition if the rule's accuracy improves without it.
- C5.0 is Quinlan's latest version release under a proprietary license. It uses less memory and builds smaller rulesets than C4.5 while being more accurate.
- CART (Classification and Regression Trees) is very similar to C4.5, but it differs in that it supports numerical target variables (regression) and does not compute rule sets. CART constructs binary trees using the feature and threshold that yield the largest information gain at each node.
- scikit-learn uses an optimized version of the CART algorithm; however, scikit-learn implementation does not support categorical variables for now.

Other python libraries (chefboost)

- However, chefboost is a python library that can be used for other decision tree algorithms.
- **ChefBoost** is a lightweight decision tree framework for Python **with categorical feature support**. It covers regular decision tree algorithms: ID3, C4.5, CART, CHAID and regression tree; also some advanced techniques: gradient boosting, random forest and adaboost. You just need to write **a few lines of code** to build decision trees with Chefboost.
- https://github.com/serengil/chefboost

Example code based on IRIS Data set

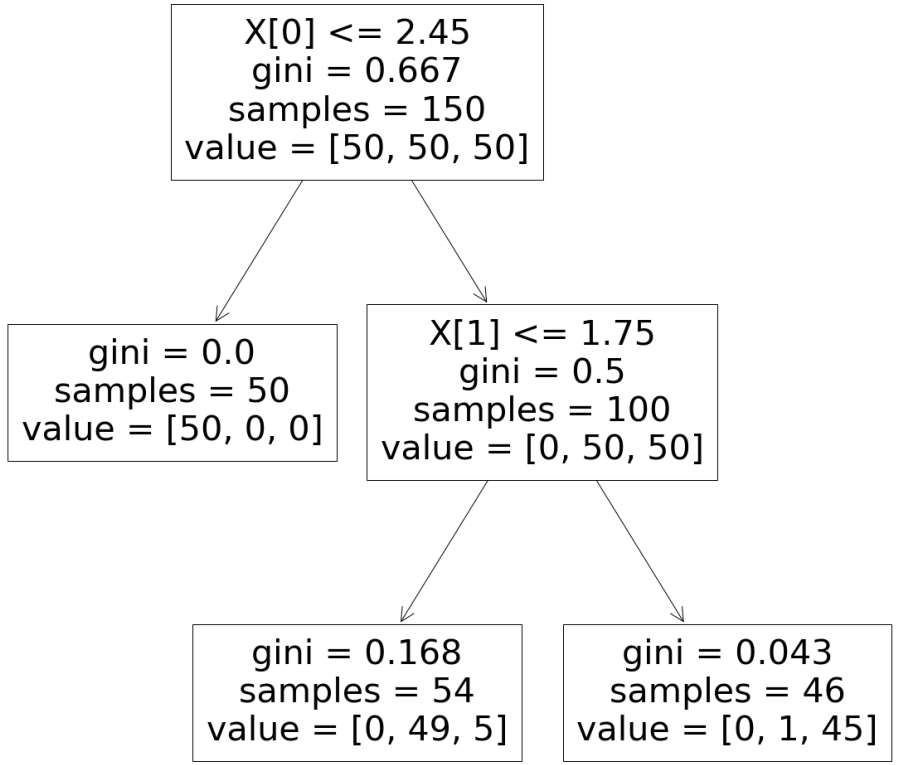
```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassi
fier

iris = load_iris()
iris['feature_names']
```

```
['sepal length (cm)', 'sepal width (cm)',
'petal length (cm)', 'petal width (cm)']
```

```
X = iris.data[:, 2:] # petal length and width
y = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2)
tree_clf.fit(X, y)
import sklearn.tree as tree
```

```
plt.figure(figsize = (20,20))
tree.plot_tree(tree_clf)
plt.show()
```

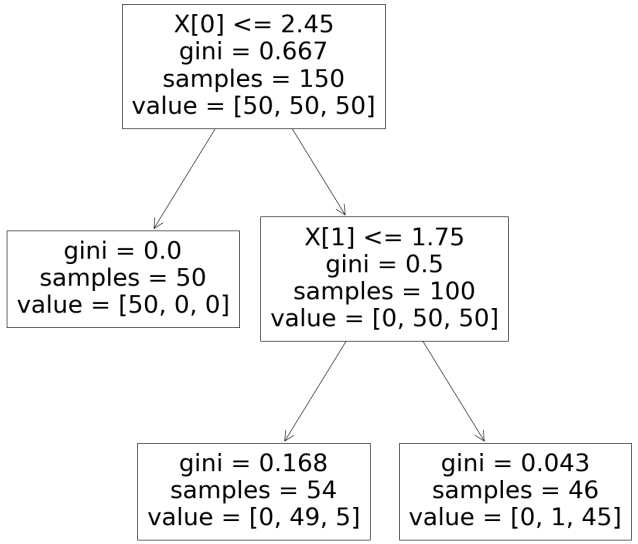


Gini index

Equation 6-1. Gini impurity

$$G_i = 1 - \sum_{k=1}^{n} {p_{i,k}}^2$$

- $p_{i,k}$ is the ratio of class k instances among the training instances in the ith node.



- For example, the depth-2 left node has a gini score equal to 1 – (0/54)2 – (49/54)2 – (5/54)2 ≈ 0.168.

Feature Scaling is not necessary and CART

- One of the many qualities of Decision Trees is that they require very little data preparation. In particular, they don’t require feature scaling or centering at all.

- Scikit-Learn uses the CART algorithm, which produces only *binary trees*: nonleaf nodes always have two children (i.e., questions only have yes/no answers). However, other algorithms such as ID3 can produce Decision Trees with nodes that have more than two children.

Decision Boundary

- Figure 6-2 shows this Decision Tree's decision boundaries for our example.
- The thick vertical line represents the decision boundary of the root node (depth 0): petal length = 2.45 cm.
- Since the left area is pure (only Iris-Setosa), it cannot be split any further.
- However, the right area is impure, so the depth-1 right node splits it at petal width = 1.75 cm (represented by the dashed line).
- Since max_depth was set to 2, the Decision Tree stops right there.
- However, if you set max_depth to 3, then the two depth-2 nodes would each add another decision boundary (represented by the dotted lines).

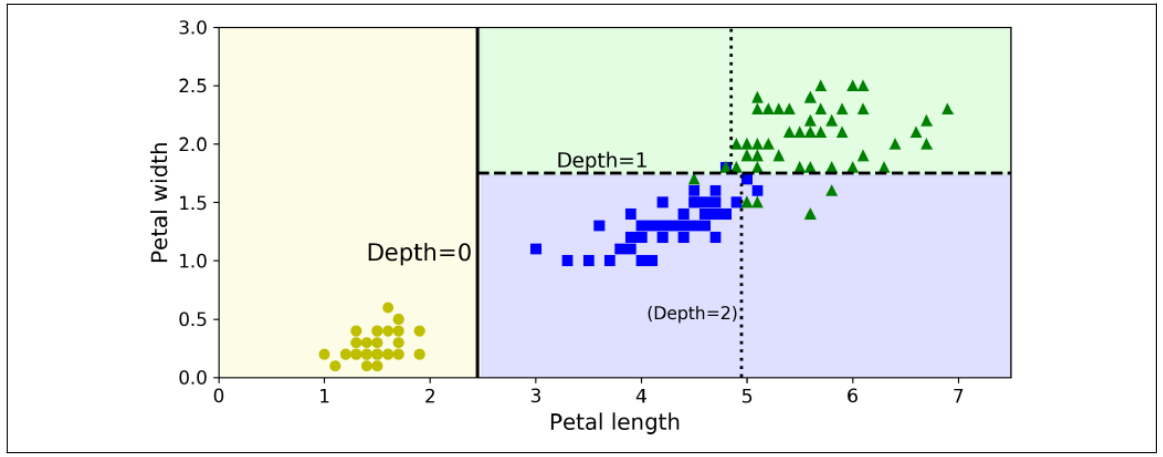
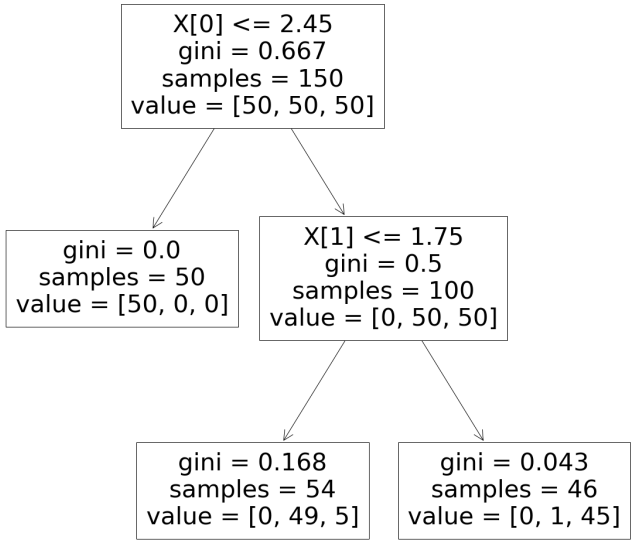


Figure 6-2. Decision Tree decision boundaries



Model Interpretation: White Box Versus Black Box

- As you can see Decision Trees are fairly intuitive and their decisions are easy to interpret. Such models are often called *white box models*.
- In contrast, as we will see, Random Forests or neural networks are generally considered *black box models*. They make great predictions, and you can easily check the calculations that they performed to make these predictions; nevertheless, it is usually hard to explain in simple terms why the predictions were made.
- For example, if a neural network says that a particular person appears on a picture, it is hard to know what actually contributed to this prediction: did the model recognize that person's eyes? Her mouth? Her nose? Her shoes? Or even the couch that she was sitting on?
- Conversely, Decision Trees provide nice and simple classification rules that can even be applied manually if need be (e.g., for flower classification).

Estimating Class Probabilities

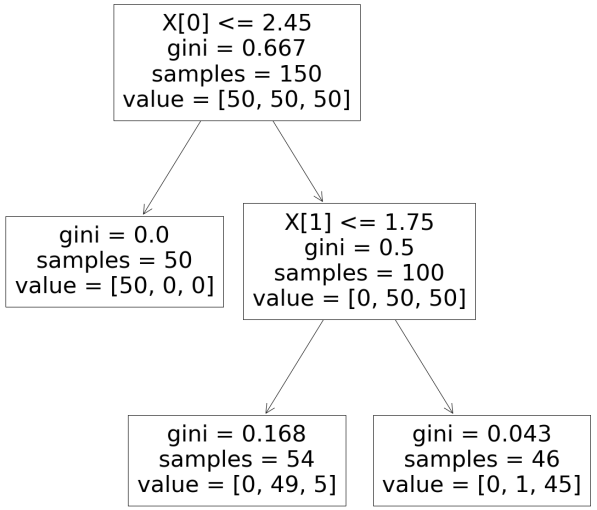
- A Decision Tree can also estimate the probability that an instance belongs to a particular class *k*: first it traverses the tree to find the leaf node for this instance, and then it returns the ratio of training instances of class *k* in this node.
- For example, suppose you have found a flower whose petals are 5 cm long and 1.5 cm wide. The corresponding leaf node is the depth-2 left node, so the Decision Tree should output the following probabilities: 0% for Iris-Setosa (0/54), 90.7% for Iris-Versicolor (49/54), and 9.3% for Iris-Virginica (5/54).
- And of course if you ask it to predict the class, it should output Iris-Versicolor (class 1) since it has the highest probability.
- Let's check this:

```
tree_clf.predict_proba([[5, 1.5]])
```

```
array([[0. , 0.90740741, 0.09259259]])
```

```
tree_clf.predict([[5, 1.5]])
```

```
array([1])
```



Gini Impurity or Entropy?

- By default, the Gini impurity measure is used, but you can select the *entropy* impurity measure instead by setting the criterion hyperparameter to "entropy".
- Both of them discusses the impurity
- The truth is, most of the time it does not make a big difference: they lead to similar trees.
- Gini impurity is slightly faster to compute, so it is a good default.
- However, when they differ, Gini impurity tends to isolate the most frequent class in its own branch of the tree, while entropy tends to produce slightly more balanced trees.

Regularization Hyperparameters

- Decision Trees make very few assumptions about the training data (as opposed to linear models, which obviously assume that the data is linear, for example).
- If left unconstrained, the tree structure will adapt itself to the training data, fitting it very closely, and most likely overfitting it.
- To avoid overfitting the training data, you need to restrict the Decision Tree's freedom during training.
- The regularization hyperparameters depend on the algorithm used, but generally you can at least restrict the maximum depth of the Decision Tree. In Scikit-Learn, this is controlled by the max_depth hyperparameter (the default value is None, which means unlimited).
- Reducing max_depth will regularize the model and thus reduce the risk of overfitting.
- The DecisionTreeClassifier class has a few other parameters that similarly restrict the shape of the Decision Tree:
  - min_samples_split (the minimum number of samples a node must have before it can be split),
  - min_samples_leaf (the minimum number of samples a leaf node must have),
  - min_weight_fraction_leaf (same as min_samples_leaf but expressed as a fraction of the total number of weighted instances)
  - max_leaf_nodes (maximum number of leaf nodes), and
  - max_features (maximum number of features that are evaluated for splitting at each node).
  - Increasing min_* hyperparameters or reducing max_* hyperparameters will regularize the model.

Decision Tree Pruning

- Other algorithms work by first training the Decision Tree without restrictions, then *pruning* (deleting) unnecessary nodes.
- A node whose children are all leaf nodes is considered unnecessary if the purity improvement it provides is not *statistically significant*.
- Standard statistical tests, such as the $\chi 2$ *test*, are used to estimate the probability that the improvement is purely the result of chance (which is called the *null hypothesis*).
- If this probability, called the *pvalue*, is higher than a given threshold (typically 5%, controlled by a hyperparameter), then the node is considered unnecessary and its children are deleted.
- The pruning continues until all unnecessary nodes have been pruned.

Example google colab notebook

- https://colab.research.google.com/drive/1fOOgxwKQRMZK5CrgFVXdUERYcGZD3Beu?usp=sharing
- https://colab.research.google.com/github/AhmedMorra1/DataSciencePractice/blob/master/Patient_Classification_Using_Decision_Trees.ipynb
- Other interesting reading:
- An example implementation of decision tree: https://www.geeksforgeeks.org/decision-tree-implementation-python/
- Could be a good reading with another step by step example: https://www.analyticsvidhya.com/blog/2020/10/all-about-decision-tree-from-scratch-with-python-implementation/

Reference

- **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by** Aurélien Géron *(Many examples are taken from this textbook)*
- **Principles of Data Mining,** Max Bramer